

Busca paralela de texto com OpenMP

Computação Paralela — TP1 — Grupo 11 — Integrantes: _____

1. Problema

A aplicação busca uma palavra em vários arquivos TXT e informa as ocorrências por arquivo e o total. A busca é literal, diferencia maiúsculas de minúsculas e não conta ocorrências sobrepostas. O tempo inclui abertura, leitura, busca e fechamento dos arquivos.

Há potencial de paralelização porque cada arquivo pode ser pesquisado independentemente. Os limites principais são o acesso ao sistema de arquivos e o desbalanceamento causado por arquivos de tamanhos diferentes.

2. Implementação OpenMP

A versão sequencial percorre os arquivos em ordem. A versão paralela aplica `parallel for` com `schedule(runtime)` e `reduction(+:total)`. A redução evita condição de corrida, enquanto `schedule(runtime)` permite comparar `static`, `dynamic` e `guided` sem recompilar.

Cada iteração escreve em uma posição exclusiva do vetor de resultados. Portanto, somente o total precisa da redução e não é necessária uma região crítica.

3. Metodologia

As medições foram feitas no nodo atlantica07 do LAD: dois Xeon E5520, oito núcleos físicos e 16 threads lógicas. O código foi compilado com GCC 9.4.0, `-O2` e `-fopenmp`. Foram usados `OMP_PROC_BIND=spread` e `OMP_PLACES=cores`.

Cada configuração teve um aquecimento e 15 execuções; a mediana foi usada por ser menos sensível a variações. Na escala forte foram mantidos 1.024 arquivos e 105,8 MB. Na fraca foram mantidos 64 arquivos e 6,61 MB por thread.

4. Resultados

As versões sequencial e paralela encontraram 786.432 ocorrências, confirmando a correção. Na escala forte, o speed-up foi 2,020 com duas threads, 4,041 com quatro, 7,171 com oito e 7,589 com 16.

O ganho foi quase linear até quatro threads e permaneceu alto nos oito núcleos físicos. De oito para 16 threads houve pouca melhora, pois as threads adicionais usam SMT e compartilham as unidades de execução, cache e largura de banda. Assim, a eficiência caiu de 89,6% para 47,4%.

5. Escalonamento e balanceamento

Foram avaliados `static`, `dynamic` e `guided` com chunks 1, 4 e 16. A menor mediana foi 26,545 ms com `dynamic,1` e 16 threads, mas `guided,1` obteve 26,653 ms. A diferença é menor que a dispersão e não permite declarar um vencedor permanente.

O chunk 16 teve distribuição perfeita em bytes; mesmo assim, nem sempre foi o mais rápido. Conteúdo, cache, NUMA e overhead também influenciam. `Dynamic` e `guided` podem compensar diferenças entre os arquivos, enquanto `static` possui menor custo de distribuição.

6. Conclusão

A paralelização preservou o resultado e explorou bem os oito núcleos físicos. A escala fraca ficou próxima do ideal até quatro threads e atingiu 87,6% de eficiência com oito. O principal limite surgiu com 16 threads, quando o SMT não duplicou os recursos físicos. Valores pouco acima de 100% foram tratados como efeitos de cache e variação experimental.

Ferramenta de IA foi usada para apoiar a organização dos testes, os cálculos estatísticos e a redação. O grupo revisou o código, as fórmulas e a interpretação dos resultados.

Resultados medidos no LAD

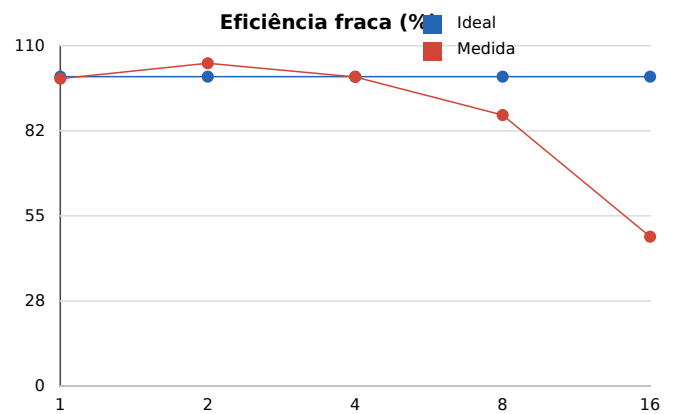
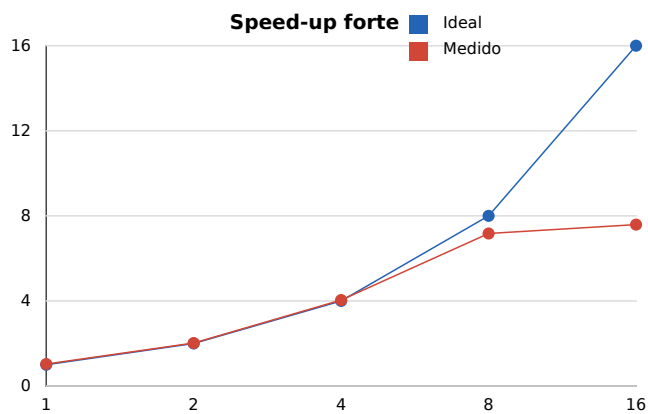
Medianas de 15 execuções

Escalabilidade forte — corpus fixo de 1.024 arquivos (static, chunk 1)

Versão	Threads	Tempo (ms)	Speed-up	Eficiência
Seq.	1	216,781	1,000	100,0%
OpenMP	1	209,847	1,033	103,3%
OpenMP	2	107,295	2,020	101,0%
OpenMP	4	53,642	4,041	101,0%
OpenMP	8	30,229	7,171	89,6%
OpenMP	16	28,565	7,589	47,4%

Escalabilidade fraca — 64 arquivos e 6,61 MB por thread (static, chunk 1)

Versão	Threads	Arquivos	Tempo (ms)	Speed-up	Eficiência
Seq.	1	64	13,700	1,000	100,0%
OpenMP	1	64	13,789	0,994	99,4%
OpenMP	2	128	13,132	2,087	104,3%
OpenMP	4	256	13,714	3,996	99,9%
OpenMP	8	512	15,641	7,007	87,6%
OpenMP	16	1024	28,379	7,724	48,3%



Melhor escalonamento observado por quantidade de threads

Threads	Configuração	Tempo (ms)	Speed-up	Eficiência
1	static,1	209,847	1,033	103,3%
2	dynamic,1	103,723	2,090	104,5%
4	guided,1	49,778	4,355	108,9%
8	guided,4	28,361	7,644	95,5%
16	dynamic,1	26,545	8,167	51,0%

Código-fonte — versão sequencial

buscador_sequencial.c

```
1 #define _DEFAULT_SOURCE
2
3 #include <dirent.h>
4 #include <inttypes.h>
5 #include <omp.h>
6 #include <stdio.h>
7 #include <stdlib.h>
8 #include <string.h>
9
10 #define DIRETORIO_TEXTOS "textos"
11
12 static int arquivo_txt(const struct dirent *entrada) {
13     size_t tamanho = strlen(entrada->d_name);
14     return tamanho > 4 &&
15         strcmp(entrada->d_name + tamanho - 4, ".txt") == 0;
16 }
17
18 static uint64_t buscar_no_arquivo(const char *caminho, const char *palavra) {
19     FILE *arquivo = fopen(caminho, "r");
20     if (arquivo == NULL) {
21         fprintf(stderr, "Nao foi possivel abrir %s\n", caminho);
22         return 0;
23     }
24
25     uint64_t ocorrencias = 0;
26     size_t tamanho_palavra = strlen(palavra);
27     char *linha = NULL;
28     size_t capacidade = 0;
29
30     while (getline(&linha, &capacidade, arquivo) != -1) {
31         char *posicao = linha;
32         while ((posicao = strstr(posicao, palavra)) != NULL) {
33             ocorrencias++;
34             posicao += tamanho_palavra;
35         }
36     }
37
38     free(linha);
39     fclose(arquivo);
40     return ocorrencias;
41 }
42
43 int main(int argc, char *argv[]) {
44     if (argc != 2 || argv[1][0] == '\0') {
45         fprintf(stderr, "Uso: %s palavra\n", argv[0]);
46         return EXIT_FAILURE;
47     }
48
49     struct dirent **arquivos;
50     int quantidade = scandir(DIRETORIO_TEXTOS, &arquivos, arquivo_txt, alphasort);
51     if (quantidade < 0) {
52         fprintf(stderr, "Nao foi possivel abrir o diretorio %s\n",
53             DIRETORIO_TEXTOS);
54         return EXIT_FAILURE;
55     }
56     if (quantidade == 0) {
57         fprintf(stderr, "Nenhum arquivo .txt encontrado em %s\n",
58             DIRETORIO_TEXTOS);
59         free(arquivos);
60         return EXIT_FAILURE;
61     }
62
63     uint64_t *ocorrencias = calloc((size_t)quantidade, sizeof(*ocorrencias));
64     if (ocorrencias == NULL) {
65         fprintf(stderr, "Memoria insuficiente\n");
66         return EXIT_FAILURE;
67     }
68
69     uint64_t total = 0;
70     double inicio = omp_get_wtime();
71
72     for (int i = 0; i < quantidade; i++) {
73         char caminho[4096];
74         snprintf(caminho, sizeof(caminho), "%s/%s",
75             DIRETORIO_TEXTOS, arquivos[i]->d_name);
76
77         ocorrencias[i] = buscar_no_arquivo(caminho, argv[1]);
78         total += ocorrencias[i];
79     }
80
81     double tempo = omp_get_wtime() - inicio;
82
83     for (int i = 0; i < quantidade; i++) {
84         printf("%s: %" PRIu64 " ocorrencia(s)\n",
85             arquivos[i]->d_name, ocorrencias[i]);
86     }
```

```
87     printf("Total: %" PRIu64 " ocorrencia(s)\n", total);
88     printf("Tempo sequencial: %.6f segundos\n", tempo);
89
90     for (int i = 0; i < quantidade; i++) {
91         free(arquivos[i]);
92     }
93     free(arquivos);
94     free(ocorrencias);
95     return EXIT_SUCCESS;
96 }
```

Código-fonte — versão paralela OpenMP

buscador_paralelo.c

```
1 #define _DEFAULT_SOURCE
2
3 #include <dirent.h>
4 #include <inttypes.h>
5 #include <omp.h>
6 #include <stdio.h>
7 #include <stdlib.h>
8 #include <string.h>
9
10 #define DIRETORIO_TEXTOS "textos"
11
12 static int arquivo_txt(const struct dirent *entrada) {
13     size_t tamanho = strlen(entrada->d_name);
14     return tamanho > 4 &&
15         strcmp(entrada->d_name + tamanho - 4, ".txt") == 0;
16 }
17
18 static uint64_t buscar_no_arquivo(const char *caminho, const char *palavra) {
19     FILE *arquivo = fopen(caminho, "r");
20     if (arquivo == NULL) {
21         fprintf(stderr, "Nao foi possivel abrir %s\n", caminho);
22         return 0;
23     }
24
25     uint64_t ocorrencias = 0;
26     size_t tamanho_palavra = strlen(palavra);
27     char *linha = NULL;
28     size_t capacidade = 0;
29
30     while (getline(&linha, &capacidade, arquivo) != -1) {
31         char *posicao = linha;
32         while ((posicao = strstr(posicao, palavra)) != NULL) {
33             ocorrencias++;
34             posicao += tamanho_palavra;
35         }
36     }
37
38     free(linha);
39     fclose(arquivo);
40     return ocorrencias;
41 }
42
43 int main(int argc, char *argv[]) {
44     if (argc != 2 || argv[1][0] == '\0') {
45         fprintf(stderr, "Uso: %s palavra\n", argv[0]);
46         return EXIT_FAILURE;
47     }
48
49     struct dirent **arquivos;
50     int quantidade = scandir(DIRETORIO_TEXTOS, &arquivos, arquivo_txt, alphasort);
51     if (quantidade < 0) {
52         fprintf(stderr, "Nao foi possivel abrir o diretorio %s\n",
53             DIRETORIO_TEXTOS);
54         return EXIT_FAILURE;
55     }
56     if (quantidade == 0) {
57         fprintf(stderr, "Nenhum arquivo .txt encontrado em %s\n",
58             DIRETORIO_TEXTOS);
59         free(arquivos);
60         return EXIT_FAILURE;
61     }
62
63     uint64_t *ocorrencias = calloc((size_t)quantidade, sizeof(*ocorrencias));
64     if (ocorrencias == NULL) {
65         fprintf(stderr, "Memoria insuficiente\n");
66         return EXIT_FAILURE;
67     }
68
69     uint64_t total = 0;
70     double inicio = omp_get_wtime();
71
72     #pragma omp parallel for schedule(runtime) reduction(+:total)
73     for (int i = 0; i < quantidade; i++) {
74         char caminho[4096];
75         snprintf(caminho, sizeof(caminho), "%s/%s",
76             DIRETORIO_TEXTOS, arquivos[i]->d_name);
77
78         ocorrencias[i] = buscar_no_arquivo(caminho, argv[1]);
79         total += ocorrencias[i];
80     }
81
82     double tempo = omp_get_wtime() - inicio;
83
84     for (int i = 0; i < quantidade; i++) {
85         printf("%s: %" PRIu64 " ocorrencia(s)\n",
86             arquivos[i]->d_name, ocorrencias[i]);
```

```
87     }
88     printf("Total: %" PRIu64 " ocorrencia(s)\n", total);
89     printf("Threads: %d\n", omp_get_max_threads());
90     printf("Tempo paralelo: %.6f segundos\n", tempo);
91
92     for (int i = 0; i < quantidade; i++) {
93         free(arquivos[i]);
94     }
95     free(arquivos);
96     free(ocorrencias);
97     return EXIT_SUCCESS;
98 }
```